



TEMA 05.5 • O DOM na Prática

Document Object Model: Como o JavaScript localiza, lê, altera, estiliza e controla a visibilidade de qualquer elemento da tela viva



1. Localizar



2. Ler



3. Modificar



4. Estilizar



5. Mostrar/Esconder



10 Questões com Gabarito



1. O que é o DOM?



A Árvore Genealógica da Página:

O **DOM (Document Object Model)** é a representação em árvore que o navegador cria para o seu HTML. Cada tag vira um **objeto vivo na memória** que o JavaScript consegue acessar e alterar em tempo real, sem recarregar o site!

document (A raiz do seu site)

└─ <html>

└─ <head> → <title>

└─ <body>

└─ <h2 id="nome">Ana Beatriz </h2>

└─ <p id="cargo">Desenvolvedora Mobile </p>

└─ <button onclick="...">Clique </button>



2. Estúdio Interativo: As 5 Capacidades do DOM

Veja como o JavaScript manipula **um único elemento com** `id="cartao-vivo"` através dos 5 botões abaixo:



Ana Beatriz

Desenvolvedora Mobile



1. Localizar



2. Ler Dados



3. Modificar Nome



4. Alternar Destaque

5.  Esconder  Resetar

Clique em um dos botões acima para ver o comando JavaScript executado em tempo real...



3. Os Comandos que Você Usará no Dia a Dia

O que você quer fazer?	Comando JavaScript	Exemplo Prático
1. Localizar elemento por ID	<code>document.getElementById("id")</code>	<pre>let c = document.getElementById("cartao") ;</pre>
2. Ler texto de tags (h1, p, div)	<code>elemento.innerText</code>	<pre>let nome = document.getElementById("nome") .innerText;</pre>
3. Ler o que foi digitado em input	<code>input.value</code>	<pre>let campo = document.getElementById("meuInput").value;</pre>
4. Alterar texto na tela	<code>elemento.innerText = "Novo"</code>	<pre>document.getElementById("nome") .innerText = "Carlos";</pre>
5. Alternar classe CSS	<code>elemento.classList.toggle("classe")</code>	<pre>c.classList.toggle("destaque") ;</pre>
6. Mostrar / Esconder	<code>elemento.style.display = "none" / "block"</code>	<pre>c.style.display = "none";</pre>



4. Códigos Práticos para Testar no OneCompiler

Códigos curtos e completos prontos para você testar no navegador:

1. Ler Input e Alterar Texto na Tela (innerText & value)

lab1-dom-texto.html

 Copiar

 OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
```

```
<title>DOM - Leitura e Escrita</title>
<style>
  body { font-family: sans-serif; text-align: center; padding: 20px; background:
#f8fafc; }
  .card { background: white; padding: 20px; border-radius: 12px; max-width:
300px; margin: 0 auto; border: 1px solid #cbd5e1; }
  input { width: 100%; padding: 8px; border-radius: 6px; border: 1px solid #ccc;
margin-bottom: 10px; }
  button { background: #0284c7; color: white; border: none; padding: 10px;
width: 100%; border-radius: 6px; font-weight: bold; cursor: pointer; }
  h3 { color: #1e1b4b; margin-top: 14px; }
</style>
</head>
<body>
  <div class="card">
    <h3 id="titulo-nome">Ana Beatriz</h3>

    <!-- 1. Input lido com .value -->
    <input type="text" id="campo-novo-nome" placeholder="Digite um novo nome">
    <button onclick="alterarNome()">Atualizar Nome</button>
  </div>

  <script>
    function alterarNome() {
      // Lê o que foi digitado com .value
      let novoNome = document.getElementById("campo-novo-nome").value;

      if (novoNome.trim() !== "") {
        // Altera o texto na tela com .innerText
        document.getElementById("titulo-nome").innerText = novoNome;
      } else {
        alert("Digite um nome primeiro!");
      }
    }
  </script>
</body>
</html>
```

2. Alternar Classes CSS e Mostrar/Esconder Elemento

lab2-dom-estilos-visibilidade.html

 Copiar

 OneCompiler

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>DOM - Classes e Visibilidade</title>
  <style>
    body { font-family: sans-serif; text-align: center; padding: 20px; background:
#f8fafc; }
    #cartao { background: white; padding: 20px; border-radius: 12px; max-width:
280px; margin: 10px auto; border: 2px solid #cbd5e1; transition: 0.3s; }

    /* Classe adicionada dinamicamente pelo JS */
```

```
.destaque { background: #fef3c7 !important; border-color: #f59e0b !important;
transform: scale(1.04); }

button { padding: 8px 14px; margin: 4px; border-radius: 6px; border: none;
font-weight: bold; cursor: pointer; }
.btn-estilo { background: #7c3aed; color: white; }
.btn-vis { background: #dc2626; color: white; }
</style>
</head>
<body>
  <div id="cartao">
    <h3>🍕 Pizza Especial</h3>
    <p>R$ 35,00</p>
  </div>

  <button class="btn-estilo" onclick="toggleEstilo()">🎨 Alternar
Destaque</button>
  <button class="btn-vis" id="btn-vis" onclick="toggleVisibilidade()">👁️
Esconder</button>

  <script>
    function toggleEstilo() {
      // Alterna a classe CSS .destaque
      document.getElementById("cartao").classList.toggle("destaque");
    }

    function toggleVisibilidade() {
      let c = document.getElementById("cartao");
      let btn = document.getElementById("btn-vis");

      if (c.style.display === "none") {
        c.style.display = "block";
        btn.innerText = "👁️ Esconder";
      } else {
        c.style.display = "none";
        btn.innerText = "👁️ Mostrar";
      }
    }
  </script>
</body>
</html>
```



5. Exercício de Fixação do DOM (10 Questões)

Responda às questões com base no funcionamento do DOM. Clique em **"Verificar Respostas"** ao final:

1 Qual é a definição correta de ****DOM (Document Object Model)****?

☐ Um antivírus pago do Windows.

☒ A representação em árvore da página HTML que permite ao JavaScript acessar e modificar os elementos em tempo real.

☐ O cabo de rede conectado ao computador.

☐ Uma folha de papel impressa.

2 Qual comando JavaScript é utilizado para **localizar** um elemento na tela pelo seu identificador exclusivo?

☐ `window.search("meu-id")`

☐ `console.find("meu-id")`

☒ `document.getElementById("meu-id")`

☐ `html.select("meu-id")`

3 Qual é a diferença fundamental entre `.value` e `.innerText` no JavaScript?

☒ `.value` lê o que o usuário digitou em campos `<input>`; `.innerText` lê ou altera o texto dentro de tags como `<h2>`, `<p>` e `<div>`.

☐ `.value` apaga a página; `.innerText` fecha o navegador.

☐ Ambos realizam exatamente a mesma operação sem nenhuma diferença.

☐ `.innerText` só funciona em computadores desktop.

4 [O que este código fará?]

Analise a instrução:

```
document.getElementById("status").innerText = "Pedido Entregue!";
```

☐ Reinicia o computador do cliente.

☐ Desliga a conexão Wi-Fi do smartphone.

☐ Apaga o botão da tela.

☒ Localiza o elemento com `id="status"` e substitui o texto visível dentro dele por "Pedido Entregue!".

5 Qual comando JavaScript é utilizado para **esconder visualmente** um elemento da tela?

☐ `elemento.style.display = "block";`

☒ `elemento.style.display = "none";`

☐ `elemento.delete();`

☐ `elemento.hideText();`

6 Para que serve o método `elemento.classList.toggle("destaque")`?

☐ Para excluir o arquivo CSS do projeto.

☐ Para multiplicar o preço do produto por dois.

☒ Para alternar a classe CSS (se ela não existir no elemento, adiciona; se já existir, remove).

☐ Para reiniciar a contagem de cliques.

7 O DOM conecta o HTML ao JavaScript, permitindo criar interfaces interativas que atualizam textos, preços e cores sem precisar recarregar a página.

☒ Verdadeiro

☐ Falso

8 Se tentarmos ler o que o usuário escreveu em uma caixa `<input id="campo">` usando `.innerText` em vez de `.value`, o resultado virá vazio porque inputs armazenam o dado digitado na propriedade `.value`.

☒ Verdadeiro

☐ Falso

9 [Trace de Código] Se executarmos `cartao.style.display = "block"`, qual estado visual o elemento assumirá na página?

☒ Ficará visível na tela.

☐ Ficará invisível/oculto.

☐ Ficará transparente com borda vermelha.

☐ Será excluído do HTML permanentemente.

10 Por que é uma boa prática usar `classList.add("destaque")` em vez de alterar 10 propriedades com `elemento.style` direto no JavaScript?

☐ Porque o JavaScript não aceita a propriedade style.

- ☐ Porque o CSS é desligado automaticamente durante o clique.
- ☐ Para obrigar o usuário a recarregar o site.
- ☒ **Porque mantém o código organizado: o CSS cuida da aparência visual na classe e o JavaScript cuida apenas da lógica de ligar/desligar.**

✓ Verificar Respostas

🔄 Limpar

Desempenho do Aluno

10 / 10

✓ **Q1: Correta!**

O DOM é a árvore de objetos gerada pelo navegador que permite ao JavaScript ler e alterar o HTML em tempo real.

✓ **Q2: Correta!**

`document.getElementById()` é o método nativo mais rápido e direto para localizar um elemento pelo seu ID único.

✓ **Q3: Correta!**

`.value` captura o que foi digitado em tags ; `.innerText` altera o texto visível dentro de tags normais (`h1`, `p`, `div`).

✓ **Q4: Correta!**

A instrução substitui o texto do elemento com `id="status"` pela string "Pedido Entregue!".

✓ **Q5: Correta!**

`style.display = "none"` oculta o elemento visualmente e remove seu espaço ocupado na tela.

✓ **Q6: Correta!**

`classList.toggle()` adiciona a classe se ela não estiver presente ou remove caso já esteja aplicada.

✓ **Q7: Correta!**

Verdadeiro. O DOM possibilita a interatividade fluida sem exigir recarregamento da página.

✓ **Q8: Correta!**

Verdadeiro. Inputs de formulário guardam seus dados na propriedade `.value`, e não em `.innerText`.

✓ **Q9: Correta!**

`style.display = "block"` torna o elemento visível como bloco normal na página.

✓ **Q10: Correta!**

Separar o visual no CSS e usar o JS apenas para alternar a classe é a melhor prática de organização de código.

 Salvar em PDF

Após testar os comandos no OneCompiler e concluir o Quiz, envie o seu relatório:



Enviar Relatório do Tema 05.5 →